```
# Printing a simple string
print("Hello World")
```

```
Hello World
```

```
# Assigning values to variables
x = 3
y = 55

# Basic Arithmetic Operations
print(x * 3)       # Multiplication
print(x + 2)       # Addition
print(x - 3)       # Subtraction
print(x ** 2)      # Exponentiation (x raised to the power of 2)

# Printing multiple operations at once
print(x, x + 2, x - 3, x ** 2, x * 2)

# Using multiple variables together
print(x + y)
```

```
9
5
0
9
3 5 0 9 6
58
```

```
# Python is case-sensitive
var = 45
VAR = 3
print(var) # Prints 45
print(VAR) # Prints 3

# Checking Data Types
a = 23
b = 23.5
c = "23.5"

print(type(a)) # <class 'int'>
print(type(b)) # <class 'float'>
print(type(c)) # <class 'str'>

# Boolean Operations and Comparisons
x = 34
y = 45
print(x > y)   # False
print(x != y)  # True

# Using 'and' / 'or' (No need for && or || like in Java)
t = True
f = False
print(t or f)  # Prints True
print(t and f) # Prints False
```

```
45
3
<class 'int'>
<class 'float'>
<class 'str'>
False
True
True
False
```

```
# Initializing a list
my_list = ["Vasiullah", "Ritesh", "Sanjana"]
print(type(my_list))

# Appending (adding) a new element to the end of the list
my_list.append("Tessica")
print(my_list)

# Indexing: Python starts counting at 0
print(my_list[1])  # Prints the 2nd name (Ritesh)
print(my_list[-1]) # Negative indexing prints from the end (Tessica)
```

```
# Modifying a list (Replacing "Vasiullah" with "Umar")
my_list[0] = "Umar"
print(my_list)

# Slicing: Extracting a subset (1:3 means from index 1 up to, but not including, index 3)
print(my_list[1:3])
```

```
<class 'list'>
['Vasiullah', 'Ritesh', 'Sanjana', 'Tessica']
Ritesh
Tessica
['Umar', 'Ritesh', 'Sanjana', 'Tessica']
['Ritesh', 'Sanjana']
```

```
# Initializing a tuple
my_l = ("Vasiullah", "Ritesh", "Sanjana")
print(type(my_l))

# Slicing works the exact same way as lists
print(my_l[1:3])

# NOTE: The following code would throw an error because tuples are immutable!
# my_l.append("Tessica")  <- AttributeError
# my_l[0] = "Rudra"       <- TypeError
```

```
<class 'tuple'>
('Ritesh', 'Sanjana')
```

```
# Initializing a set with deliberate duplicates
my_s = {"Mango", "mango", "Watermelon", "Guava", "Pineapple", "mango"}

# When printed, duplicates are removed, but case-sensitivity matters
# ("Mango" and "mango" are treated as two different items)
print(my_s)

# NOTE: You cannot use indexing or slicing on sets because they are unordered!
# print(my_s[0]) <- TypeError: 'set' object is not subscriptable
```

```
{'Watermelon', 'Mango', 'Guava', 'mango', 'Pineapple'}
```

```
Var1 = {"Mango","Banana","Pineapple","Apple","apple","Apple","Mango"}
Var2 = {"Apple","Banana","Grapes","Grapes","Kiwi","Kiwi","Orange"}
### union of sets
Var1.union(Var2)
```

```
{'Apple', 'Banana', 'Grapes', 'Kiwi', 'Mango', 'Orange', 'Pineapple', 'apple'}
```

```
### Intersection of sets
Var1.intersection(Var2)
```

```
{'Apple', 'Banana'}
```

```
### Difference
Var1.difference(Var2)
```

```
{'Mango', 'Pineapple', 'apple'}
```

```
Var2.difference(Var1)
```

```
{'Grapes', 'Kiwi', 'Orange'}
```

```
Var1.add("Kiwi")
```

```
Var1
```

```
{'Apple', 'Banana', 'Kiwi', 'Mango', 'Pineapple', 'apple'}
```

```
Batsman = ["Sachin","Saurav","Sehwag","Virat","Dhoni"]
```

```
Bowlers = ["Bumrah","Arshdeep","Shami","Archer","Siraj"]
```

```
Batsman = ["Sachin","Saurav","Sehwag","Virat","Dhoni"]
Bowlers = ["Bumrah","Arshdeep","Shami","Archer","Siraj"]
Allplayers = Batsman+Bowlers
```

```
Allplayers
```

```
['Sachin',
 'Saurav',
 'Sehwag',
 'Virat',
 'Dhoni',
 'Bumrah',
 'Arshdeep',
 'Shami',
 'Archer',
 'Siraj']
```

Double-click (or enter) to edit

```
Allplayers = Batsman + Bowlers
Allplayers.remove("Siraj")
```

Start coding or generate with AI.

```
# write the multiplication from 2 to 10 using nested loop

for num in range(2, 11):
    print("Multiplication of ", num)
    print(".............................")

    for i in range(1, 11):
        result = num * i
        print(f"{num} x {i} = {result}")
```

```
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
```

```
10 x 7 = 70
10 x 8 = 80
10 x 9 = 90
10 x 10 = 100
```

```
### zip function

players = ["Virat", "Aiyer", "Rajat", "David", "Kronel"]

Runs = [120, 100, 130, 199, 123]

Age = [32, 33, 35, 25, 28]

Todays_Score = [10, 35, 100, 101, 120]

Var_mapped = zip(players, Runs, Age, Todays_Score)

print(list(Var_mapped))
```

```
[('Virat', 120, 32, 10), ('Aiyer', 100, 33, 35), ('Rajat', 130, 35, 100), ('David', 199, 25, 101), ('Kronel', 123, 28, 120)]
```

```
players = ["Virat", "Aiyer", "Rajat", "David", "Kronel"]
Runs = [120, 100, 130, 199, 123]
Age = [32, 33, 35, 25, 28]
Todays_Score = [10, 35, 100, 101, 120]

Var_mapped = zip(players, Runs, Age, Todays_Score)
P, R, A, T = zip(*Var_mapped)
```

```
All_planets = [
    ["Mercury", "Venus", "Earth"],
    ["Mars", "Jupiter", "Saturn"],
    ["Uranus", "Neptune", "Pluto"]
]

print(All_planets)
```

```
[['Mercury', 'Venus', 'Earth'],
 ['Mars', 'Jupiter', 'Saturn'],
 ['Uranus', 'Neptune', 'Pluto']]
```

```
single_list = []

for sublist in All_planets:
    for planet in sublist:
        if len(planet) < 6:
            single_list.append(planet)

print(single_list)
```

```
['Venus', 'Earth', 'Mars', 'Pluto']
```

```
### single of code using list comprehension

#var = [expression outer loop inner loop condition ]

singleList = [planet for sublist in All_planets for planet in sublist if len(planet)>6]

singleList
```

```
['Mercury', 'Jupiter', 'Neptune']
```

```
# A tuple containing strings, a list, floats, and integers
var_3 = ("Ram", "Sham", [2, 3, 4, 5], 25, 25.5)

# We cannot do var_3[0] = "Ria" because the tuple itself is immutable.
# BUT, we can target the list at index 2, and change its elements!

# Targeting the 2nd element (index 1) of the list (which is at index 2 of the tuple)
var_3[2][1] = 20
print(var_3) # The inner list becomes [2, 20, 4, 5]
```

```
('Ram', 'Sham', [2, 20, 4, 5], 25, 25.5)
```

```
# IMPLICIT CONVERSION
a = 20      # Integer
b = 20.5    # Float
c = a + b   # Python automatically converts 'a' to a float so the math works
```

```
print(c)    # 40.5 (Float)

# EXPLICIT CONVERSION
p = "10"    # String
q = 5       # Integer

# We cannot do 'p + q' here. Python won't add a string and a number.
# We must explicitly convert the string to an integer first:
r = int(p) + q
print(r)    # Prints 15

# Alternatively, convert the integer to a string to concatenate them:
s = p + str(q)
print(s)    # Prints "105"
```

```
40.5
15
105
```

```
# Concatenating strings
st1 = "I am learning Python"
st2 = "for Machine Learning"

# Adding a space between the strings
full_string = st1 + " " + st2
print(full_string)

# Getting the length of a string (Spaces are counted!)
print(len(st1))
```

```
I am learning Python for Machine Learning
20
```

```
# --- Basic If/Else & Type Conversion ---
# Taking user input and immediately converting the string to an integer
num = int(input("Enter your number: "))

if num > 0:
    print("The number is positive")
elif num < 0:
    print("The number is negative")
else:
    print("The number is zero")

# --- Nested If Statements ---
# Checking multiple conditions inside one another
if num > 0:
    print("The number is positive")
    # Using the modulus operator (%) to check for a remainder
    if num % 2 == 0:
        print("The number is even")
    else:
        print("The number is odd")

# --- The Grading System (Multiple Elif Statements) ---
marks = int(input("Enter your marks: "))

if marks < 60:
    print("Your grade is F")
elif marks < 70:
    print("Your grade is D")
elif marks < 80:
    print("Your grade is C")
elif marks < 90:
    print("Your grade is B")
else:
    print("Your grade is A")
```

```
Enter your number: 90
The number is positive
The number is positive
The number is even
Enter your marks: 92
Your grade is A
```

```
# --- Basic For Loop ---
my_marks = [86, 89, 77, 78, 76]
total_sum = 0

for mark in my_marks:
    # Adding 5 grace marks to each score and printing
    print("With grace marks:", mark + 5)
```

```
    # Adding to our running total
    total_sum = total_sum + mark

# This print statement is OUTSIDE the loop because it is not indented
print("Total sum of marks:", total_sum)

# --- Using Enumerate ---
students = ["Afifa", "Saket", "Yuvaraj", "Srivardhan"]

# Enumerate automatically generates an index number for each item
for index, student in enumerate(students):
    print("Index:", index, "| Name:", student)
```

```
With grace marks: 91
With grace marks: 94
With grace marks: 82
With grace marks: 83
With grace marks: 81
Total sum of marks: 406
Index: 0 | Name: Afifa
Index: 1 | Name: Saket
Index: 2 | Name: Yuvaraj
Index: 3 | Name: Srivardhan
```

```
# --- While Loop ---
count = 0

while count < 5:
    print("Count is:", count)
    # The incrementing statement is required to prevent an infinite loop!
    count = count + 1

print("The loop ends here.")

# --- Loop Control Statements ---
word = "Machine Learning"

# 1. BREAK: Stops the loop entirely as soon as the condition is met
print("\n--- Testing Break ---")
for letter in word:
    if letter == "L":
        print("Found L! Breaking out of loop.")
        break
    print(letter)

# 2. CONTINUE: Skips the current iteration but keeps the loop running
print("\n--- Testing Continue ---")
for letter in word:
    if letter == "L":
        continue # Skips printing "L" but moves to the next letter
    print(letter)

# 3. PASS: Does absolutely nothing. Acts as a placeholder.
print("\n--- Testing Pass ---")
for letter in word:
    if letter == "L":
        pass # The code just passes right over this
    print(letter)
```

```
Count is: 0
Count is: 1
Count is: 2
Count is: 3
Count is: 4
The loop ends here.

--- Testing Break ---
M
a
c
h
i
n
e

Found L! Breaking out of loop.

--- Testing Continue ---
M
a
c
h
i
n
```

```
e

e
a
r
n
i
n
g

--- Testing Pass ---
M
a
c
h
i
n
e

L
e
a
r
n
i
n
g
```

```
# Take user input and convert to integer
num = int(input("Enter the number: "))

print(f"\nMultiplication table for {num}:")
# range(1, 11) loops from 1 up to, but NOT including, 11
for i in range(1, 11):
    result = num * i
    # Using f-string formatting to print the equation
    print(f"{num} x {i} = {result}")
```

```
Enter the number: 89

Multiplication table for 89:
89 x 1 = 89
89 x 2 = 178
89 x 3 = 267
89 x 4 = 356
89 x 5 = 445
89 x 6 = 534
89 x 7 = 623
89 x 8 = 712
89 x 9 = 801
89 x 10 = 890
```

```
# Outer loop: Iterates through numbers 2 to 10
for num in range(2, 11):
    print(f"\n--- Multiplication of {num} ---")

    # Inner loop: Iterates 1 to 10 for the current 'num'
    for i in range(1, 11):
        result = num * i
        print(f"{num} x {i} = {result}")
```

```

--- Multiplication of 2 ---
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

--- Multiplication of 3 ---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30

--- Multiplication of 4 ---
```

```
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40

--- Multiplication of 5 ---
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

--- Multiplication of 6 ---
6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
```

```
# Individual lists
players = ["Virat", "Rajat", "David", "Krunal"]
runs = [120, 130, 90, 123]
ages = [32, 35, 25, 28]
todays_score = [10, 100, 101, 120]

# Zipping them together
var_mapped = zip(players, runs, ages, todays_score)

# Converting to a list to print (This consumes/empties the zip object!)
mapped_list = list(var_mapped)
print("Zipped List:")
print(mapped_list)

# UNZIPPING
# Because we emptied the zip object above, we must zip the list using the unpack operator (*)
p, r, a, t = zip(*mapped_list)

print("\nUnzipped Tuples:")
print("Players:", p)
print("Runs:", r)
```

```
Zipped List:
[('Virat', 120, 32, 10), ('Rajat', 130, 35, 100), ('David', 90, 25, 101), ('Krunal', 123, 28, 120)]

Unzipped Tuples:
Players: ('Virat', 'Rajat', 'David', 'Krunal')
Runs: (120, 130, 90, 123)
```

```
# 2D List (A list containing 3 sublists)
all_planets = [
    ["Mercury", "Venus", "Earth"],
    ["Mars", "Jupiter", "Saturn"],
    ["Uranus", "Neptune", "Pluto"]
]

single_list = []

# Outer loop cycles through the 3 sublists
for sublist in all_planets:
    # Inner loop cycles through the planets inside the current sublist
    for planet in sublist:
        # Filtering condition: Only append if length is greater than 6
        if len(planet) > 6:
            single_list.append(planet)

print("Nested Loops Result:", single_list)
```

```
Nested Loops Result: ['Mercury', 'Jupiter', 'Neptune']
```

```
# Syntax: [expression  outer_loop  inner_loop  condition]
comp_list = [planet for sublist in all_planets for planet in sublist if len(planet) > 6]
```

```
print("List Comprehension Result:", comp_list)
```

```
List Comprehension Result: ['Mercury', 'Jupiter', 'Neptune']
```

```
marks = [24, 25, 26, 27]

# Adding 2 to ALL marks
all_bonus = [x + 2 for x in marks]
print("All Bonus:", all_bonus)

# Adding 2 ONLY to even marks
even_bonus = [x + 2 for x in marks if x % 2 == 0]
print("Even Bonus:", even_bonus)
```

```
All Bonus: [26, 27, 28, 29]
Even Bonus: [26, 28]
```

```
Start coding or generate with AI.
```